

Garmin Ransomware Attack and Lessons for Software Engineering

<https://www.wired.com/story/garmin-outage-ransomware-attack-workouts-aviation/>

The article we chose to summarize is the “A Cyberattack on Garmin Disrupted More Than Workouts” from WIRED that explains how Garmin was affected in July 2020 by a ransomware attack. Garmin is commonly known for their fitness watches and wearable devices that seem to only have affected people trying to track health statistics and workouts, however, Garmin also works with important aviation services that made the attack much more severe. This represents how an attack is a good example of how the failure of one company’s computer systems can spread into many different areas.

The attack caused Garmin Connect, their website, customer support, and email systems to shut down and have outages for multiple days, preventing users from syncing their fitness and health information from Garmin wearable devices. More importantly, however, Garmin Pilot and flyGarmin experienced outages, and pilots depend on these services for flight planning, navigation, and various database updates. One of the major issues described in the article involved the aeronautical databases used by Garmin navigation equipment. These databases have to remain constantly updated for certain types of flying which is shown by the Front Range Flight School in Colorado example, where one aircraft was temporarily grounded because its Garmin navigation database could not be updated. Pilots could use other methods for some tasks, such as filing a flight plan directly through the FAA, but the convenience and automation normally provided by Garmin's systems was missing. Garmin's problem with their computer network eventually affected pilots, flight schools, aircraft, fitness customers, customer support, and much more. Garmin began restoring its systems about four days after the outage started, and the company stated that it had no indication that customer information, including Garmin Pay information, had been accessed, lost, or stolen.

Reports cited by WIRED indicated that the attack involved WastedLocker ransomware, which had been associated with the cybercriminal organization known as Evil Corp. Ransomware is dangerous because attackers disable an organization's systems and demand payment in order to restore access, and the article emphasized how these attacks have become especially threatening to organizations that provide important services because attackers know that these organizations experience significant pressure to restore their systems quickly. This also

represents the need for ample backup systems within large companies. A security expert quoted in the article explains that ransomware attacks expose organizations that don't have adequate fallback systems available, and although pilots are taught to have "backups of backups," pilots still have to expect that systems may eventually fail. This attack on Garmin is more than a cybersecurity incident, and instead, it demonstrates how dependent modern day systems have become on technology. That is why a failure inside one part of the system can cause unexpected and much larger failures throughout the bigger environment or system within a company.

We think the Garmin ransomware attack has several important connections to software engineering because modern day software rarely operates completely by itself. Most applications depend on databases, cloud services, APIs, authentication systems, networks, etc. and because of this, software engineers have to think beyond whether or not their individual code works correctly. They have to think about what happens when another part of the entire system stops working or if one thing going offline will become a much larger inconvenience. Although Garmin Connect going offline only seems to inconvenience uploading workouts, it's actually related to the aviation services that affected pilots' convenient access to necessary services. The importance of scope within software development and systems is crucial especially in large companies like Garmin, because the software system can change based on its related and dependent other systems as it did in this example. It was more surprising to our group that Garmin had this issue in the first place, because we'd only ever known them for their watches, but it was concerning to know that they also controlled aviation services when hearing about this attack. The Garmin incident also shows why redundancy and fallback procedures are important. A well-designed system should avoid having a single point of failure whenever the system performs an important function. Identifying the dependencies is crucial in the design process, because databases becoming unavailable, APIs not responding, and even authentication failures are all important factors to consider when designing a product that has this wide range of users. For less important applications, an outage may only be annoying. For systems used in aviation, healthcare, transportation, banking, or infrastructure, an outage can have much more serious consequences.

Testing is another important lesson from the Garmin attack. Software teams usually test whether their systems operate correctly under normal circumstances, but engineers should also test how systems behave when things go wrong. This could include intentionally disconnecting

databases, shutting down servers, disabling APIs, or simulating a cyberattack. A related concept is a system of creating a digital twin, and running cyberattacks or pushing errors on it to see how the duplicate system responds to them. The goal would be to see whether the system recovers properly or whether one failure causes the entire system to collapse. Relating off of that, cybersecurity should be a large consideration of software quality rather than an afterthought or something added after development. A system may have great features and minimal software bugs but still be susceptible to attacks if cybersecurity features aren't implemented. The Garmin case is a good example of that, as they were targeted by a large cyberattack and didn't have any prior ability to prevent or mitigate the issue.

There is also an ethical responsibility involved in this situation, because software engineers may work on systems that thousands or millions of people depend on. A developer may only see one small feature, but that feature may eventually become part of a much larger system. Engineers therefore have a responsibility to communicate serious risks instead of ignoring them because it could potentially affect a large population of people who depend on those systems for everyday use. Again, for a large company like Garmin, it wasn't as big of an issue that users couldn't log their workouts or use their watch, it was more for their services like flyGarmin or Garmin Pilot. A ransomware attack against one company did not only affect that company's computers but it affected athletes, customer support systems, pilots, flight schools, and aircraft operations. Software engineers should expect these kinds of dependencies instead of assuming that each system operates independently.

Overall, the biggest lesson from the Garmin attack is that engineers need to design systems with failure in mind, because it is impossible to guarantee that a server will never fail, a software will never contain a bug, or an organization will never experience a cyberattack. Instead, however, software engineers can prepare for these issues and create systems that can detect the failures within the system they are designing in case anything were to go wrong. This would limit the failure's impact, allow for quicker recovery, and provide alternative various options when something goes wrong. Garmin's outage shows that as software becomes more connected to the physical world, a software or cybersecurity failure can quickly become much more than a computer problem.